

Cálculo Paralelo do Conjunto de Mandelbrot com MPI + OpenMP

Versão híbrida: MPI Coordenador/Trabalhador entre nós e *workpool* OpenMP (sem mestre) dentro do nó

Bernardo Luiz Padilha Zamin • João Pedro Sostruznik Sotero da Cunha
Programação Paralela — Trabalho 4 (MPI + OpenMP), PUCRS — Escola Politécnica, 2026

1. Introdução

Implementa-se, em C, a versão **híbrida MPI + OpenMP** do cálculo do **conjunto de Mandelbrot** (imagem 7680×4320), reaproveitando o *Coordenador/Trabalhador* do grupo. São **dois níveis de paralelismo**: o **MPI** distribui blocos de linhas entre os **nós** (**um processo pesado por nó**) e o **OpenMP** usa os núcleos *dentro* de cada nó no modelo **workpool sem mestre**. Avaliam-se *speed-up*, eficiência e escalabilidade **forte** e **fraca** em **4 nós**, o efeito do **Hyper-Threading**, as **formas de alocar o coordenador** e a **comparação com a MPI pura**.

2. Modelo híbrido

Entre nós (MPI). Modelo **Coordenador/Trabalhador idêntico** ao da MPI pura. O rank 0 distribui **blocos de rows_per_task linhas sob demanda**: o trabalhador pede (TAG_REQUEST), recebe TAG_TASK ou TAG_STOP, calcula e devolve (TAG_RESULT). Há **um processo pesado por nó**, com todos os núcleos à disposição das threads. Para cada pixel itera-se $z \leftarrow z^2 + c$ até $|z| > 2$ ou `max_iter`; pontos *internos* custam `max_iter` e *externos* terminam cedo, gerando **carga irregular** que exige **balanceamento dinâmico**.

3. Workpool OpenMP sem mestre

Cada bloco é processado por uma **região paralela** (`#pragma omp parallel`). O controle está numa **estrutura de dados compartilhada** `WorkPool{next,total}`: **todas as threads** executam o mesmo laço e **retiram atômica**mente a próxima linha (`#pragma omp atomic capture`) até esvaziar o pool — **sem thread mestre**, com escalonamento *self-service* emergindo da estrutura de dados. A granularidade é de **uma linha** (7680 px). Usa-se `MPI_Init_thread` (`MPI_THREAD_FUNNELED`) e `OMP_NUM_THREADS` de 1 a 16 (até 2 threads por núcleo, HT).

4. Alocação do coordenador

O coordenador só distribui/coleta, ficando **quase ocioso**. Por isso adota-se **sempre** a alocação `-N 4 -n 5`: 4 nós, **1 coordenador + 4 trabalhadores**, com o coordenador *compartilhando* o nó 0 (**sem dedicar um nó a ele**). Como comparação, mede-se também a alocação *dedicada* `-N 4 -n 4` (coordenador sozinho num nó, só 3 trabalhadores).

5. Metodologia

4 nós da Atlantica (LAD), SLURM, Xeon E5520, **16 CPUs lógicas/nó** (8 núcleos $\times$ 2 HT). `ladcomp -env mpicc -O3`

`-fopenmp`. Tempo por `MPI_Wtime` *antes* do I/O. Baseline $T(1) = 93,19$ s (sequencial, `max_iter=2000`). O híbrido roda **sempre em `-N 4 -n 5`** e a escala se dá pelo **nº de threads por trabalhador** (`OMP_NUM_THREADS` $\in \{1, 2, 4, 8, 16\}$, ou seja **4 a 64 núcleos** de cálculo = 4 trab. $\times$ threads). **Forte**: `max_iter=2000` fixo. **Fraca**: `max_iter=1500` $\times$ threads (carga por núcleo constante). $\text{Speed-up} = T(1)/T(p)$; $\text{Eficiência} = \text{Speed-up}/p$, $p = 4 \times \text{threads}$. MPI pura: 1 processo por unidade (32 = 1/núcleo; 64 = 2/núcleo, HT). Saída **byte-idêntica** à sequencial (md5).

6. Resultados e análise

Escalabilidade forte (Tab. 1, Fig. a). Com a alocação MPI fixa e aumentando as threads dos 4 trabalhadores, o speed-up cresce de **4,3** $\times$ (4 núcleos) a **46,3** $\times$ (64 núcleos). A eficiência fica próxima do ideal até 16 núcleos ($\approx 1,0$) e cai a **0,72** em 64, quando entram os fluxos de **Hyper-Threading** (2 por núcleo físico) e há leve oversubscrição no nó do coordenador. A pequena *superlinearidade* com poucas threads ($E > 1$) vem do menor *working-set* por núcleo (melhor uso de cache) ante o baseline sequencial de 1 núcleo.

Híbrido $\times$ MPI pura (Tab. 3, Fig. b). Na mesma alocação de 4 nós, o híbrido **praticamente empata** com a MPI pura: em 64 unidades, **2,01 s** (46,3 $\times$) vs 2,00 s (46,5 $\times$); em 32 unidades, 3,31 s vs 3,20 s. Distribuir por *threads* (híbrido) ou por *processos* (MPI pura) rende igual neste problema *compute-bound* — mas o híbrido usa **muito menos ranks MPI** (5 vs 64) e menos memória/mensagens, vantagem que cresce com a escala.

Alocação do coordenador (Tab. 4, Fig. c). Dedicar um nó inteiro ao coordenador ocioso (n4, 3 trabalhadores) desperdiça 1/4 da máquina: 2,96 s. *Compartilhar* seu nó (n5, 4 trabalhadores) usa os 4 nós e cai para **2,01 s — 32% mais rápido**. Por isso a alocação padrão é n5.

Escalabilidade fraca (Tab. 2, Fig. d). Com a carga proporcional às threads (`max_iter=1500` $\times$ threads), o tempo fica **praticamente constante** até 32 núcleos (16,6 $\rightarrow$ 17,7 s, eficiência **0,94**) e sobe a 22,2 s em 64 (0,75), de novo pela região de HT. Ou seja, escalabilidade fraca quase ideal na faixa de núcleos físicos.

7. Conclusão

A versão cumpre o modelo: **MPI Coordenador/Trabalhador** (um processo por nó) + **workpool OpenMP sem mestre**. Seguindo a metodologia de `-N 4 -n 5` escalando as threads, o híbrido atinge **46,3** $\times$ (64 núcleos), **empatado com a MPI pura**, com escalabilidade fraca quase ideal até os núcleos físicos. **Compartilhar o nó do coordenador (n5) é 32% melhor que dedicá-lo (n4)**. A eficiência cai na região de Hyper-Threading, como esperado.

Tabelas e gráfico

Tabela 1 — Escalabilidade forte (híbrido N4 n5)

threads	p	Tempo (s)	Speed-up	Efic.
1	4	21,89	4,26	1,06
2	8	10,98	8,49	1,06
4	16	5,86	15,9	0,99
8	32	3,31	28,1	0,88
16	64	2,01	46,3	0,72

Tabela 2 — Escalabilidade fraca (N4 n5)

threads	max_iter	Tempo (s)	Efic.
1	1500	16,59	1,00
2	3000	16,34	1,02
4	6000	16,88	0,98
8	12000	17,75	0,94
16	24000	22,18	0,75

Tabela 3 — Híbrido × MPI pura (4 nós, max_iter=2000)

Versão	Unidades	Tempo (s)	Speed-up
Híbrido (16 threads/trab.)	64 threads	2,01	46,3
MPI pura (2 proc/núcleo, HT)	64 processos	2,00	46,5
Híbrido (8 threads/trab.)	32 threads	3,31	28,1
MPI pura (1 proc/núcleo)	32 processos	3,20	29,1

Tabela 4 — Alocação do coordenador (4 nós, 16 threads/trab.)

Alocação	Proc.	Trab.	Tempo (s)	Speed-up
<i>Compartilhado</i> (-N 4 -n 5)	5	4	2,01	46,3
<i>Dedicado</i> (-N 4 -n 4)	4	3	2,96	31,5

$T(1) = 93,19$ s (sequencial, 1 núcleo, max_iter=2000); Speed-up = $T(1)/T(p)$; Eficiência = Speed-up/ p , com $p = 4 \times$ threads (núcleos de cálculo = 4 trabalhadores $\times$ threads). Imagem 7680 $\times$ 4320. Escalabilidade fraca: eficiência = $T(1 \text{ thread})/T(p)$.

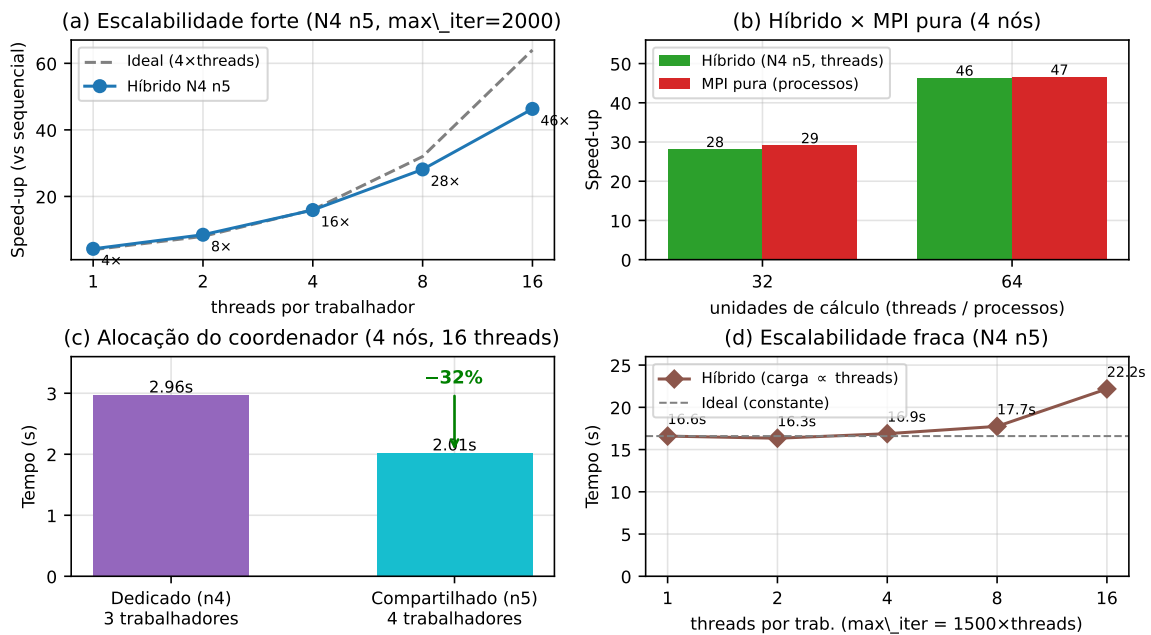


Figura 1 — (a) escalabilidade forte (speed-up $\times$ threads, N4 n5); (b) híbrido × MPI pura em 4 nós; (c) alocação do coordenador; (d) escalabilidade fraca.

Anexo — Código-fonte (mandelbrot-hib.c, trechos relevantes)

Versão híbrida MPI + OpenMP. O coordenador (rank 0) distribui blocos de linhas sob demanda e coleta os resultados (idêntico ao mandelbrot-mpi.c); cada trabalhador calcula seu bloco com o *workpool* OpenMP (compute_task_workpool, sem mestre). As funções inalteradas (mandelbrot, save_ppm) e o laço do coordenador são omitidos por brevidade. Compilação: `ladcomp -env mpicc -O3 mandelbrot-hib.c -o mandelbrot-hib -fopenmp -lm`.

```
1 #include <mpi.h>
2 #include <omp.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <math.h>
6 #define WIDTH 7680
7 #define HEIGHT 4320
8 #define DEFAULT_ROWS_PER_TASK 64
9 #define TAG_REQUEST 0
10 #define TAG_TASK 1
11 #define TAG_RESULT 2
12 #define TAG_STOP 3
13 typedef struct { int start_row; int num_rows; } Task;
14 /* Estrutura de dados compartilhada que CONTROLA o workpool (sem mestre):
15    next = índice da próxima linha do bloco ainda não tomada. */
16 typedef struct { int next; int total; } WorkPool;
17 int mandelbrot(double real, double imag, int max_iter); /* z=z^2+c; igual ao seq/MPI */
18 void save_ppm(const char *f, int *img, int max_iter); /* igual ao seq/MPI */
19 /* ===== WORKPOOL OpenMP: todas as threads trabalham, SEM MESTRE ===== */
20 void compute_task_workpool(const Task *task, int *buffer, int max_iter)
21 {
22     WorkPool pool = { .next = 0, .total = task->num_rows };
23     #pragma omp parallel /* todas as threads entram e trabalham */
24     {
25         while (1) {
26             int row;
27             #pragma omp atomic capture /* retira atômica a próxima linha */
28             { row = pool.next; pool.next++; } /* do pool compartilhado (sem mestre) */
29             if (row >= pool.total) break; /* pool vazio: esta thread termina */
30             int y = task->start_row + row;
31             for (int x = 0; x < WIDTH; x++) {
32                 double real = -2.5 + (3.5 * x / WIDTH);
33                 double imag = -1.0 + (2.0 * y / HEIGHT);
34                 buffer[row * WIDTH + x] = mandelbrot(real, imag, max_iter);
35             }
36         }
37     }
38 }
39 int main(int argc, char *argv[])
40 {
41     int max_iter = atoi(argv[1]);
42     int rows_per_task = (argc >= 3) ? atoi(argv[2]) : DEFAULT_ROWS_PER_TASK;
43     int provided; /* so a thread principal chama MPI */
44     MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
45     int rank, size;
46     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
47     MPI_Comm_size(MPI_COMM_WORLD, &size);
48     if (rank == 0) {
49         /* COORDENADOR: distribui blocos sob demanda (TAG_TASK) e coleta os
50            resultados (TAG_RESULT); cronometra (MPI_Wtime) e grava a imagem.
51            Idêntico ao mandelbrot-mpi.c (laço MPI_Probe / MPI_Recv / MPI_Send). */
52     } else {
53         /* TRABALHADOR: pede tarefa, calcula com o workpool, devolve. */
54         MPI_Status st; int request = 1;
55         while (1) {
56             MPI_Send(&request, 1, MPI_INT, 0, TAG_REQUEST, MPI_COMM_WORLD);
57             Task task;
58             MPI_Recv(&task, sizeof(Task), MPI_BYTE, 0, MPI_ANY_TAG,
59                    MPI_COMM_WORLD, &st);
60             if (st.MPI_TAG == TAG_STOP) break;
61             int *buffer = malloc(sizeof(int) * task.num_rows * WIDTH);
62             compute_task_workpool(&task, buffer, max_iter); /* <= OpenMP */
63             MPI_Send(&task, sizeof(Task), MPI_BYTE, 0, TAG_RESULT, MPI_COMM_WORLD);
64             MPI_Send(buffer, task.num_rows * WIDTH, MPI_INT, 0, TAG_RESULT,
65                    MPI_COMM_WORLD);
66             free(buffer);
67         }
68     }
69     MPI_Finalize();
70     return 0;
71 }
```